

Extending AssetOpsBench: Smart Grid Transformer Integration and Automated Scenario Generation

Integrated Smart Grid Transformer into AssetOpsBench with IEC-grounded diagnostic tools and built a Scenario Generator Agent that automates scenario creation for any new asset class, applying optimizations to achieve 8x speedup over baseline with no loss in scenario quality.

Team 8

Rohith Kanathur · rk3443

Sagar Chethan Kumar · sc5758

Ananya Kapoor · ak5447

Dev Bahl · db3791

GitHub: <https://github.com/Rohith-Kanathur/AssetOpsBench>

Tracking: <https://wandb.ai/rk3443-columbia-university/assetopsbench>

Motivation & Problem

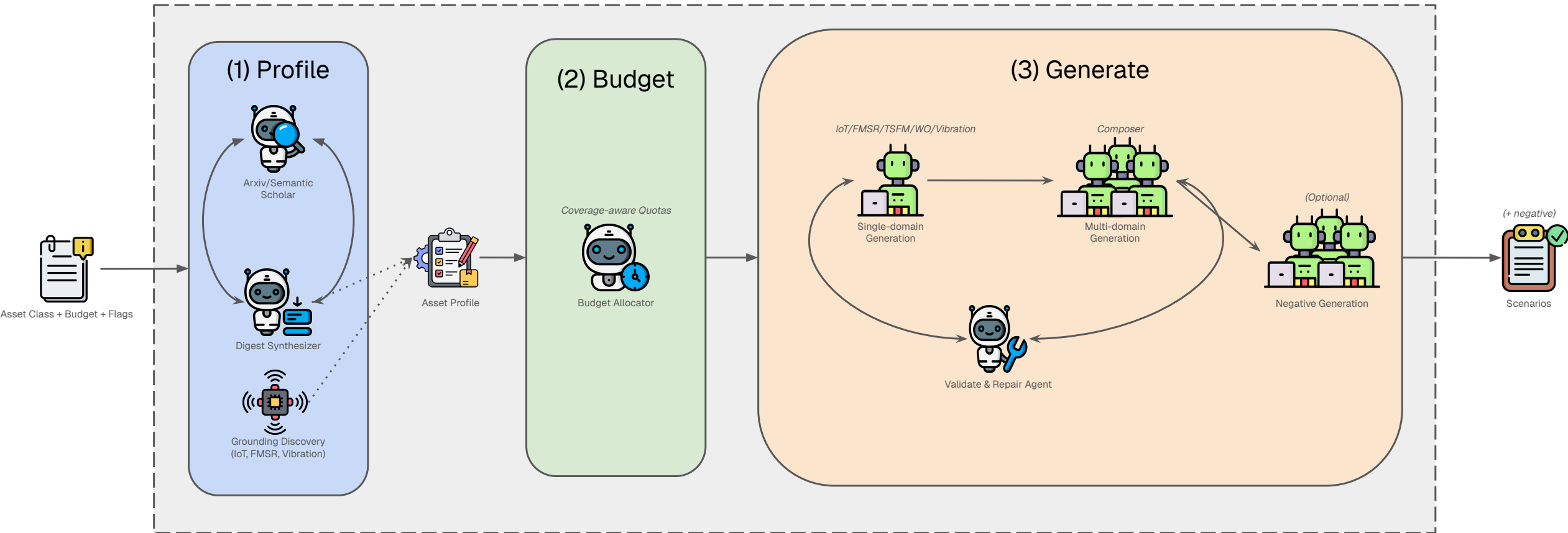
Why this matters

- Industrial asset operations depend on continuous monitoring across heterogeneous data: IoT telemetry, failure-mode libraries, oil chemistry readings, and maintenance logs.
- Evaluating LLM-based agents on these tasks requires realistic, standards-compliant scenarios across diverse asset classes and lifecycle phases.
- Manual scenario authoring by subject-matter experts (SMEs) is slow, inconsistent, and cannot scale to new asset classes.

Problem statement

Extending AssetOpsBench to new industrial assets requires manually authoring physically plausible, standards-compliant, tool-reachable evaluation scenarios, a bottleneck that blocks benchmark coverage and delays agent evaluation.

Goal: *Automate high-fidelity scenario generation for any new asset class, while applying optimization techniques such as caching, thread pool offloading and parallel scenario generation to achieve a speedup of 8× over the unoptimized baseline, with no degradation in composite scenario quality.*



Technical Approach - Model & Data

Model

- **watsonx/meta-llama/llama-3-3-70b-instruct (IBM WatsonX)** is the model used.
- **This is used as the LLM backbone in the ScenarioGeneratorAgent:** stages: asset profiling, budgeted allocation, domain wise generation, validation and repair
- **This also serves as the LLM backbone for the transformer FMSR tools:** four MCP tools grounded in IEC 60599 and IEC 60076-7 standards; health index prediction, dissolved gas analysis, load profile analysis and winding temperature analysis.

Dataset

- Existing AssetOpsBench scenarios (chillers, AHUs) as few-shot examples for generation scenario.
- ArXiv/SemanticScholar literature retrieval (domain papers fetched per asset class to build AssetProfiles for scenario generation).
- Mendeley transformer health dataset (Arias Velasquez & Lara, 2020) for the supporting health index model.

Hardware

- **Profiling:** PyTorch Profiler chrome traces and Weights & Biases for all scenario generation and evaluation runs.
- **Software stack:** Python, FastMCP (MCP Servers), asyncio, CouchDB, Weights & Biases, Pytorch Profiler

Technical Approach - Optimizations Applied

1. Caching

Asset profiles, research papers, and few-shot examples are cached at two levels: fast in-memory LRU and persistent disk storage. This avoids re-running expensive LLM calls and academic retrieval for the same asset across repeated runs.

2. Parallel Per-Domain Scenario Generation

Scenario generation across different domains (IoT, FMSR, Work Order, etc.) runs in parallel rather than sequentially.

3. Thread Pool Offloading

Blocking LLM HTTP calls are offloaded to a shared thread pool executor, keeping the async event loop free. This ensures multiple pipeline phases can make progress simultaneously without stalling on I/O.

Technical Approach - Optimizations Applied

4. Batch LLM Calls

Instead of one LLM call per scenario, multiple scenarios are requested in a single prompt. This amortizes the per-call overhead (network, parsing, rate-limiter tokens) across a batch.

5. Early Rejection Filtering

A pre-screen eliminates clearly malformed or duplicate scenarios before the full validator runs. This avoids wasting time on structural validation for outputs that fail basic length, field presence, or near-duplicate checks.

Default params used (unless specified): --max-concurrent 3 --max-workers 4 --cache-ttl 24h · --retriever arxiv

Profiling Methodology

Tools

- **PyTorch Profiler / torch.profiler**
 - All “per phase” times are wall-clock seconds measured via timed section spans and were verified against the PyTorch Profiler chrome trace.
 - Conducted four profiling experiments comparing baseline and optimized scenario generation for the Transformer asset. Each experiment isolates a specific optimization dimension: scalability, caching, parallelism, and overall combined effect.
- **Weights & Biases for run-level tracking**
 - Logging is done for scenario **generation** and **evaluation** stages.
 - Metrics captured: phase wise timing, llm call metrics, cache hit/miss metrics, valid/invalid scenario count and scenario evaluation scores.

Metrics captured (Generation)

- Per-LLM-Call Timeline (**llm/***): Wall clock time per LLM call, prompt token count, completion token count, total token count.
- Run-Level Summary Scalars:
 - **run/***: top-level metadata: asset name, generation mode (open form / closed form) and total scenarios generated.
 - **tokens/***: aggregate token usage: prompt, completion, and combined tokens across all LLM calls.
 - **timing/***: wall-clock and CPU time for each pipeline phase.
 - **cache/***: cache hit/miss counts, hit/miss rates.
 - **batch summary/***: total scenarios generated, valid, repaired, rejected, attempt count, and overall validation pass rate.

Profiling Methodology

Metrics captured

- **Rich Tables**
 - **llm calls table**: One row per LLM call with columns: phase, prompt tokens, completion tokens, total tokens, wall sec.
 - **batch results table**: One row per batch scenario generation attempt with columns: focus, attempt, generated, valid, repaired, rejected counts and pass rate.
- **Per-Scenario Timeline (eval/*)**
 - **eval/scenario_id**: Scenario identifier
 - **eval/type**: Scenario type (e.g. iot, tsfm)
 - **eval/static_score**: Static score for the scenario (/20)
 - **eval/llm score**: LLM judge score for the scenario (/30)
 - **eval/dry_run_score**: Dry-run execution score for a scenario(/50)
 - **eval/quality_score**: Composite quality score for scenario(/100)

Metrics captured (Evaluation)

- **Run-Level Summary Scalars (wandb.summary)**
 - **summary/static/***: mean static score across all scenarios
 - **summary/llm/***: mean LLM as a judge score across all scenarios
 - **summary/dry_run/***: mean dry-run score across all scenarios
 - **summary/quality/***: mean, min, and max composite quality score across all scenarios.
- **Rich Tables**
 - **eval_report_table**: one row per scenario with columns: id, type, category, evaluation scores, issues
 - **llm_calls_table**: one row per LLM call with columns: phase, prompt tokens, completion tokens, total tokens, wall sec.

Results — Headline Numbers

8x

Speedup from all optimizations

[baseline → optimized, 408 sec → 50 sec]

4.6x

Speedup from Caching

[Cold Cache → Warm Cache, 302.4 sec → 65.09 sec]

1.85x

Speedup from Parallel Scenario Generation

[C = 1 → C = 5, 87.3 sec → 47.3 sec]

All measurements are averaged over 3 runs.

C controls how many of the 5 domain {iot, fmsr, tsfm, wo, vibration} scenarios are generated in parallel.

Times listed in this slide refer to the total pipeline time for the scenario generation: transformer asset class, 50 scenarios

Results — Before vs. After (varying N = scenarios)

Baseline (Unoptimized) (Time in seconds)

Phase	N=10	N=25	N=50
Phase 0: Get Tool Desc	2.4	2.5	2.2
Phase 1: Build Asset Profile	448.2	338.9	325.7
Phase 2: Allocate Budget	2.05	3.3	2.3
Phase 3: Single Agent Gen	11.4	34.7	38.8
Phase 4: Multi Agent Gen	4.05	20.6	29.3
Total (in seconds)	468.2	400.3	408.4

Optimized (with warm cache) (Time in seconds)

Phase	N=10	N=25	N=50
Phase 0: Get Tool Desc	1.5	2.4	2.1
Phase 1: Build Asset Profile	0	0	0
Phase 2: Allocate Budget	0	0	0
Phase 3: Single Agent Gen	8.2	14.8	15.9
Phase 4: Multi Agent Gen	10.7	17.2	18.1
Total (in seconds)	13.6	19.8	50.8

- Tables report per-phase and total pipeline times for the baseline and the optimized variants.
- Caching optimization contributes the most improvement. Phase 1 dominates baseline which is **80-96% of total (almost 6 to 8 mins)**. Caching eliminates it entirely on warm runs as shown in the table on the right.

Results — Before vs. After (varying C = parallel domains)

Pipeline Time vs. Number of Domains (Time in seconds)

Phase	C=1	C=2	C=3	C=5
Phase 0: Get Tool Desc	2.18	2.15	2.18	2.45
Phase 1: Build Asset Profile	0	0	0	0
Phase 2: Allocate Budget	0	0	0	0
Phase 3: Single Agent Gen	43.88	22.29	17.66	17.62
Phase 4: Multi Agent Gen	46.07	54.51	44.44	27.22
Total (in seconds)	87.39	78.95	64.29	47.30

C controls how many of the 5 domains {iot, fmsr, tsfm, wo, vibration} scenarios are generated in parallel.

Total speedup: **1.85X** from C = 1 → C = 5.

Phase 3 speedup: **43.88 sec (C=1) to 17.62 sec (C=5)**

Cold vs Warm Cache (Optimized Variant) (Time in seconds)

Phase	Cold	Warm
Phase 0: Get Tool Desc	2.4	2.2
Phase 1: Build Asset Profile	247.4	0
Phase 2: Allocate Budget	0	0
Phase 3: Single Agent Gen	17.30	17.23
Phase 4: Multi Agent Gen	35.34	45.57
Total (in seconds)	302.47	65.09

We compare two runs of the optimized variant with identical parameters (N=50, C=3) differing only in cache state.

The cold run forces a full asset profile rebuild, the warm run reads the profile in the cache. The disk cache eliminates Phase 1 entirely on warm runs (**247.4 s to 0 s**), saving over **four minutes** per run. The full pipeline drops from **302.5 s to 65.1 s: a 4.6X speedup**

Results — Before vs. After

Pipeline time (seconds): Baseline vs. Optimized, Both Cold Cache.

Phase	Baseline	Optimized
Phase 0: Get Tool Desc	2.20	2.42
Phase 1: Build Asset Profile	325.70	216.85
Phase 2: Allocate Budget	2.34	2.56
Phase 3: Single Agent Gen	38.83	14.65
Phase 4: Multi Agent Gen	39.92	32.46
Total	408.43	268.96

Phase 1

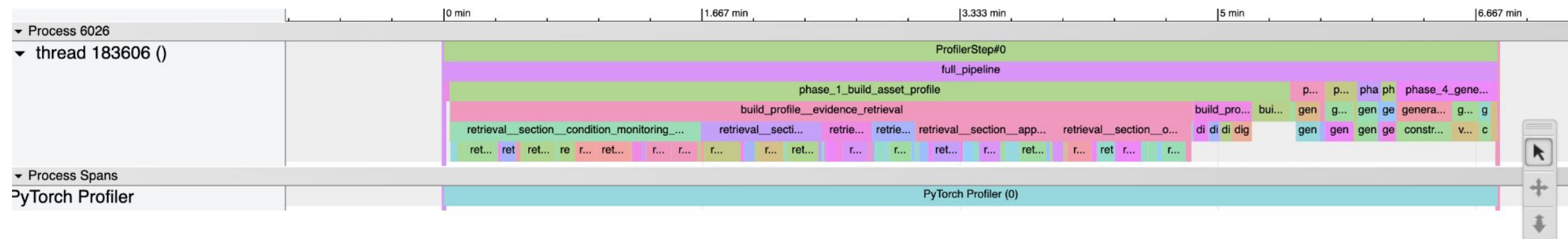
I/O concurrency (thread pool) → 33% faster even without cache

Phase 3

Phase 3 shows the largest relative gain: a 2.65× speedup from parallel focus execution

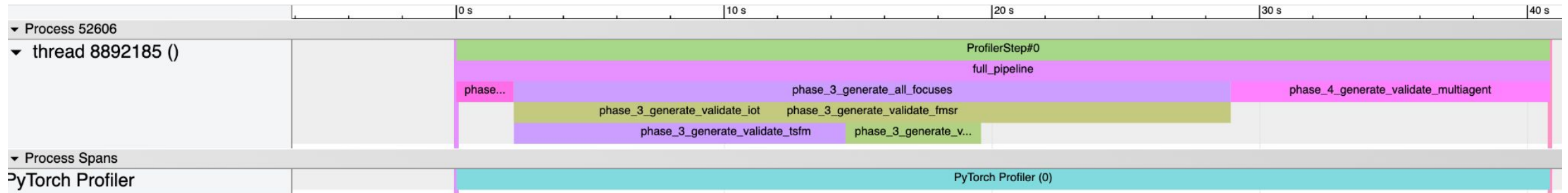
*Phase 1 improves by 33% (325.7 s → 216.9 s) through I/O concurrency.
The baseline executes PDF downloads, database queries, and literature searches sequentially.
The optimized module offloads each blocking call to a background thread pool, filling CPU-idle network-wait time with concurrent requests.*

Profiler Evidence (Baseline)



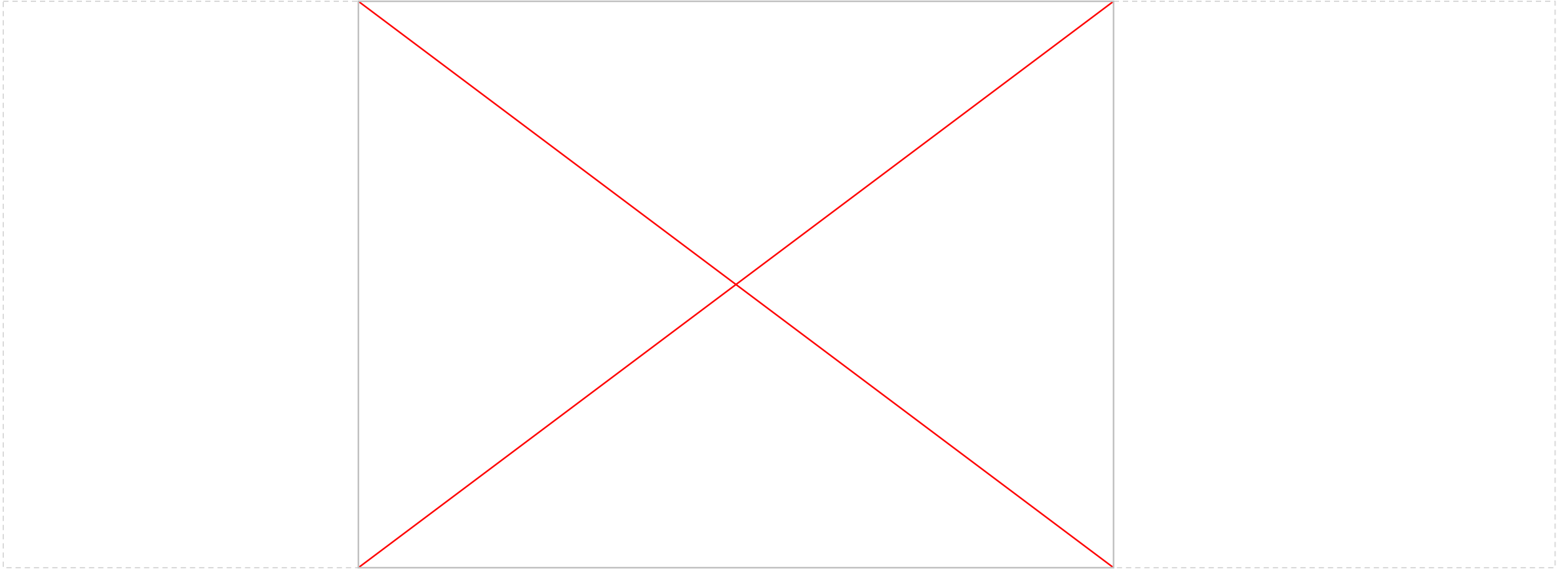
- The total pipeline time for generating 50 scenarios for transformer asset using the baseline variant is over 6.5 minutes.
- Phase 1 dominates the most taking over 5 minutes. This is due to uncached academic retrieval, PDF fetching, and expensive LLM calls on every run.

Profiler Evidence (Optimized)



- The total pipeline time is reduced to just 40 sec after applying optimizations such as parallel scenario generation, caching and thread pool offloading.
- The phase 1 time which was the bottleneck in the baseline is now reduced to 0 sec.

Demo / Visualization



Lessons Learned

What worked

- **Insight 1 (Caching Impact):** Implementing a two-level caching system (L1 in-memory LRU and L2 persistent disk storage) was the most significant optimization, eliminating Phase 1 entirely on warm runs and yielding a 4.6x speedup on its own.
- **Insight 2 (Concurrency Sweet Spot):** Parallelizing the scenario generation across different focus groups via `asyncio.gather` yielded a 2.65x speedup for Phase 3, with a concurrency limit of 3 proving to be the optimal sweet spot. Offloading blocking I/O calls to a background thread pool also successfully filled CPU-idle network-wait time with concurrent requests.
- **Insight 3 (Preserved Quality):** Achieving a 8x end-to-end speedup did not come at the cost of evaluation fidelity, as the optimized pipeline maintained a composite quality score of 74.2, which is statistically indistinguishable from the baseline which has a quality score of 73.8.

What didn't work

- **Optimization that regressed performance:** Phase 3 (single agent scenario generation) did not show improvements beyond 3 parallel tasks.
- **Tool / approach we abandoned:** We initially relied on a single LLM call to generate scenarios directly from a prompt. While the outputs were linguistically fluent, they frequently violated physical constraints, referenced unavailable tools making them unusable. This led us to introduce the validate-and-repair loop, where a second LLM pass checks each scenario and fixes issues before it is accepted.
- **Misconception we corrected mid-project:** We initially assumed that academic literature and general domain knowledge about transformers would be sufficient context for the LLM to generate valid scenarios. In practice, the generated scenarios frequently referenced sensors, asset IDs, and operating ranges that did not match the actual data in CouchDB, making them unexecutable by the agent. Grounding generation directly in live CouchDB data with real asset IDs, real sensor names, and real historical readings produced scenarios the agent could actually execute, which was reflected higher dry-run scores.

Next Steps

Broaden Asset-Class Coverage

Apply the scenario generation pipeline to additional Industry 4.0 asset classes, such as wind turbines, motors, pumps, and process equipment, to test its generality and surface domain-specific validator requirements.

Dedicated Evaluation for Negative Tracks

Develop a dedicated evaluation protocol, distinct from the positive track, for the negative-track scenarios to better probe agent refusal and insufficiency behavior under deliberate non-answerability.

Generator Fine Tuning

Explore preference-based or reinforcement-style fine-tuning of the generator itself using the validate-and-repair loop as a natural substrate. This could further reduce repair iterations and improve the first-pass yield.

Bonus Opportunities

- Open source Pull Request Submitted to AssetOpsBench:
<https://github.com/IBM/AssetOpsBench/pull/288>
- The scenarios generated from our pipeline are available on Huggingface:
<https://huggingface.co/datasets/rohithkanathur/assetopsbench-transformer-scenarios-dataset>

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Rohith Kanathur	Transformer Asset Class Integration, Profiling, Optimization, Wandb Logging, Evaluation, Documentation and Report Writing.	https://github.com/Rohith-Kanathur/AssetOpsBench/tree/feat/scenario-generator-profiling src/scenarios_{profiling, optimization, wandb}/*
Sagar Chethan Kumar	Scenario generation pipeline implementation (research & digest agent, validate & repair agent, and domain generation).	https://github.com/Rohith-Kanathur/AssetOpsBench/tree/feat/scenario-generator src/scenarios/*, docs/report.tex
Ananya Kapoor	Scenario schema validation, eval unit tests, category alignment fix, --static-only mode, literature survey and report writing.	src/scenarios_evaluation/scenario.schema.json, scripts/validate_scenarios.py, src/scenarios_evaluation/tests/ (local)
Dev Bahl	Eval tests and report rendering, baseline vs optimized comparison, CI pipeline, experiment reproduction.	src/scenarios_evaluation/tests/, scripts/render_eval_report.py, scripts/compare_eval_reports.py, .github/workflows/ scripts/reproduce_experiments.py (local)

Collaboration: Weekly team meetings via Gmeet, communications over Whatsapp. Updated mentor: Dr. Dhaval Patel weekly over email chain and overleaf documents.

Thank you

Questions?

GitHub: <https://github.com/Rohith-Kanathur/AssetOpsBench>

Tracking: <https://wandb.ai/rk3443-columbia-university/assetopsbench>

Backup — Evaluation Methodology

Stage 1

20 pts

Static Checks

- Schema completeness
- Type diversity ($\leq 50\%$ per type)
- Category alignment
- Text length (15–400 chars)
- Dedup detection

Stage 2

30 pts

LLM Judge

- Answerability (10 pts)
- Tool usability (8 pts)
- Difficulty (6 pts)
- Characteristic quality (4 pts)
- Clarity (2 pts)

Stage 3

50 Pts

Dry-Run Execution

- Plausible response (20 pts)
- Correct tools used (15 pts)
- No obvious errors (15 pts)
- Run by PlanExecuteRunner
- Highest weight: real-world signal

$Quality\ Score = Static + LLM + Dry\ Run\ Scores \in [0, 100]$
 ≥ 70 : high quality
 < 50 : low quality

Backup — Evaluation Results (N=50, 3 Runs)

Scenario quality evaluation: baseline vs. optimised (mean $\pm$ std, n = 3 runs)

Metric	Baseline	Optimized
Static Score (/20)	18.4 $\pm$ 0.6	19.1 $\pm$ 0.3
LLM Judge Score (/30)	22.2 $\pm$ 3.6	22.4 $\pm$ 1.8
Dry-Run Score (/50)	33.1 $\pm$ 1.3	32.7 $\pm$ 2.5
Quality Score (/100)	73.8 $\pm$ 3.0	74.2 $\pm$ 1.9

- Quality preserved**
0.4 point gap between baseline & optimised is within run-to-run variance
- Static scores high**
>= 18.4/20 in both schema validity unaffected by optimisations
- Dry-run comparable**
33.1 vs 32.7: agent executable quality fully preserved

Both pipelines exceed the 70-pt high-quality threshold, optimised pipeline is faster without any quality trade-off.

Backup — Wandb Screenshots

runs.summary["llm_calls_table"]

⚙

≡ Filter

	phase	prompt_tokens	completion_tokens	total_tokens	wall_s
8	phase_3_generate_fmsr	7134	153	7287	2.254
9	phase_3_repair_fmsr	6276	152	6428	2.251
10	phase_4_generate_validate_multi:	11552	1265	12817	14.84
11	phase_3_repair_multiagent	6696	365	7061	4.013
12	phase_4_generate_validate_multi:	11552	356	11908	4.331
13	phase_3_repair_multiagent	6438	238	6676	3.169

Backup — Wandb Screenshots

⋮

runs.summary["batch_results_table"]

⚙

Filter

	focus	attempt	generated	valid	repaired	rejected	pass_rate
1	iot	1	10	10	0	0	1
2	tsfm	1	10	8	2	0	1
3	fmsr	1	10	10	0	0	1
4	wo	1	5	5	0	0	1
5	fmsr	2	2	1	0	1	0.5
6	fmsr	3	1	0	1	0	1

⋮

1

- 6 of 8

⏪

⏩

Export as CSV

Columns...

Reset table

Backup — Wandb Screenshots



Backup — Wandb Screenshots

Config

View raw data

Config parameters are your model's inputs. [Learn more](#)

Q Search keys with regex

▼ Config parameters: {} 8 keys

asset_name: "Transformer"

batch_size: 10

data_in_couchdb: true

disk_cache_enabled: true

max_concurrent: 3

model_id: "watsonx/meta-llama/llama-4-maverick-17b-128e-instruct-fp8"

num_scenarios: 50

retriever: "arxiv"

Summary

View raw data

Summary metrics are your model's outputs. [Learn more](#)

Q Search keys with regex

cache/profile_l1_hit_rate: 0

cache/profile_l1_hits: 0

cache/profile_l1_misses: 2

llm_calls_table: "table-file"

llm/completion_tokens: 238

llm/phase: "phase_3_repair_multiagent"

llm/prompt_tokens: 6,438

llm/total_tokens: 6,676

llm/wall_seconds: 3.169

run/asset_name: "Transformer"

run/generation_mode: "open_form"

run/total_llm_calls: 13

run/total_scenarios: 50

46 ... 86

tokens/phase_4_generate_validate_multiagent/total: 24,725

tokens/total: 105,052

tokens/total_completion: 6,546

HPML · Spring 2026

14 / 14

Backup — Wandb Screenshots

runs.summary["eval_report_table"]

Filter

	id	type	category	static_score	llm_score	dry_run_score	quality_score	static_issues	llm_suggestion	dry_run_issues
1	transformer_sc	iot	Knowledge	20	30	15	65	none	None	Steps 2 and 5
2	transformer_sc	iot	Knowledge	20	30	35.1	85.1	none	None	step 2 used
3	transformer_sc	iot	Data Query	20	22	35	77	none	The task ..	Steps 2 and 4
4	transformer_sc	iot	Knowledge	20	30	7.5	57.5	none	None	["step 2 used
5	transformer_sc	iot	Knowledge	20	30	11.3	61.3	none	None	["Step 2 used
6	transformer_sc	iot	Data Query	20	30	42.4	92.4	none	None	Step 2 used
7	transformer_sc	iot	Knowledge	20	30	15	65	none	None	The final
8	transformer_sc	iot	Data Query	20	30	46.2	96.2	none	None	step 1 used
9	transformer_sc	fmsr	Diagnostic	20	30	44	94	none	None	Step 5 used
10	transformer_sc	fmsr	Diagnostic	20	30	50	100	none	None	None
11	transformer_sc	fmsr	Recommendation	20	30	44.8	94.8	none	None	Step 6 used
12	transformer_sc	fmsr	Diagnostic	20	30	15	65	none	None	The final
13	transformer_sc	fmsr	Diagnostic	20	30	44	94	none	None	Step 5 used

1

13 of 50

Export as CSV

Columns...

Reset table

Backup — Wandb Screenshots

runs.summary["llm_calls_table"]

Filter

	phase	prompt_tokens	completion_tokens	total_tokens	wall_s
40	judge	477	68	545	1.022
41	judge	482	62	544	1.024
42	judge	497	63	560	0.921
43	judge	504	57	561	0.924
44	judge	515	59	574	1.102
45	judge	490	66	556	1.037
46	judge	494	63	557	1.126
47	judge	522	54	576	0.819
48	judge	493	59	552	0.817
49	judge	511	63	574	1.435
50	judge	515	72	587	1.527
51	dry_run	4344	266	4610	3.132
52	dry_run	222	8	230	0.333

40 - 52 of 400

Export as CSVColumns...Reset table

Backup — Transformer Asset Class Integration

```
(assetopsbench-mcp) rohithkanathur@Mac AssetOpsBench % uv run plan-execute \  
"Assess the load profile for transformer with the following readings: \  
Voltages: VL1 110, VL2 115, VL3 108, \  
Currents: IL1 100, IL2 105, IL3 95, \  
Line-to-line voltages: VL12 225, VL23 220, VL31 218, \  
Neutral current: 5, \  
Rated MVA: 100"
```

Answer

The transformer's load profile is normal with a load of 20.5 MVA, representing 20.5% of its rated capacity. The current imbalance is relatively low at 5.26%, and the neutral current is within acceptable limits. The recommended action is to continue monitoring the transformer's loading condition.

```
(assetopsbench-mcp) rohithkanathur@Mac AssetOpsBench % uv run plan-execute "List all the sensors for transformer 1"
```

Answer

The sensors for Transformer 1 are:

1. acetylene
2. ati
3. co
4. co2
5. dbds
6. dielectric_rigidity
7. ethane
8. ethylene
9. hydrogen
10. il1
11. il2
12. il3

Backup — Transformer Asset Class Integration

```
(assetopsbench-mcp) rohithkanathur@Mac AssetOpsBench % uv run plan-execute "What is the health index of a transformer with the following sensor readings: hydrogen 2845 ppm, oxygen 5860 ppm, nitrogen 27842 ppm, methane 7406 ppm, CO 32 ppm, CO2 1344 ppm, ethylene 16684 ppm, ethane 5467 ppm, acetylene 7 ppm, DBDS 19 ppm, power factor 1, interfacial voltage 45, dielectric rigidity 55, and water content 0."
```

Answer

The health index of the transformer is 72.39, indicating a "Good" condition.

```
(assetopsbench-mcp) rohithkanathur@Mac AssetOpsBench % uv run plan-execute "Interpret the DGA gas readings for transformer: Hydrogen 100 ppm, Methane 50 ppm, Acetylene 5 ppm, Ethylene 20 ppm, Ethane 10 ppm."
```

Answer

The DGA gas readings for the transformer indicate a fault type of Low-Energy Discharge (LED) with medium confidence. The recommended action is to perform a thorough inspection and monitoring of the transformer, checking for signs of discharge or overheating, and consider further testing or maintenance to prevent potential failures.

```
(assetopsbench-mcp) rohithkanathur@Mac AssetOpsBench % uv run plan-execute "Assess the winding temperature for transformer 'TX1' with WTI 80, OTI 85, ATI 78, OTI_A 2, OTI_T 1."
```

Answer

The winding temperature assessment for transformer 'TX1' indicates a "Normal" thermal status, but with a "Critical" risk level due to an active trip flag, suggesting potential overheating or internal issues. The recommended action is immediate shutdown and inspection to prevent further damage. Key metrics include a hot spot rise of 2.0°C and an ageing rate of 1.039.